

9 things to know about software testing on agile teams



9 things to know about
software testing on agile teams

Executive Summary

Agile practices are changing how organizations approach software testing. Agile teams need continuous feedback from testing to release frequently without compromising quality. This has increased reliance on unit and interface/API testing by developers and raised questions about the role of dedicated software testers. Research by Ranorex reveals the effect of agile

practices on the role of the tester, and describes how software testers can adapt and contribute to continuous testing environments.

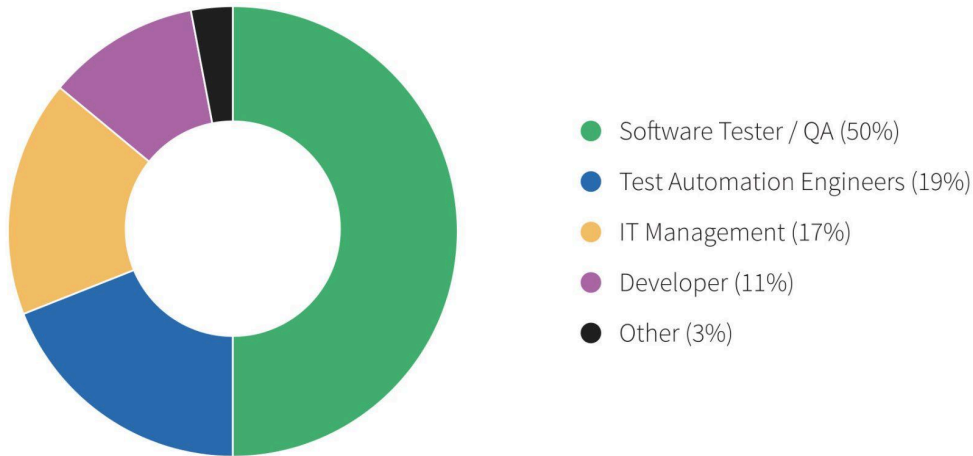
1. The transition from waterfall to agile development is still in progress.

The profession of software testing is undergoing rapid change, driven by a transformation in software development practice. Until recently, the “waterfall” model of software development has been the leading practice. According to [Gartner, Inc.](#), “waterfall methods were employed on 56% of development efforts in 2015, with iterative methods used in 21% and agile in 23%.”

Now, agile teams are in the majority. In a recent online survey, just over 80% of professional developers reported using agile methods, compared to 27% using waterfall methods. The proportion of agile development was even higher in an email survey of over 230 testing professionals conducted by Ranorex in August 2017, where 90% of respondents reported following agile practices. However, the transition to agile is incomplete: 8% of respondents to the Ranorex survey describe their development practice as still “primarily waterfall,” while another 35% are using a combination of agile and waterfall practices.

Ranorex Survey: Agile vs Waterfall

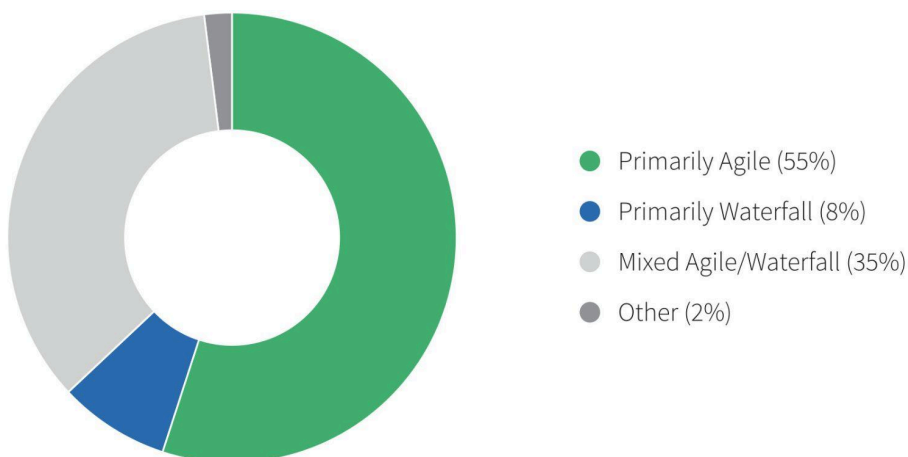
Survey Respondents



In the Ranorex survey, 50% of respondents identified as software testers/QA professionals, and another 19% as test automation engineers. 17% reported roles in IT management, including titles such as Test Manager or QA Manager.

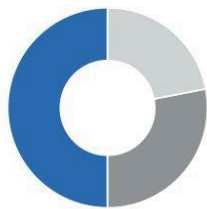
Ranorex Survey: Agile vs Waterfall

Development Environment



In a survey of over 230 testing professionals, 55% reported that they are “primarily agile,” with 35% in “mixed agile/waterfall” environments. Only 8% identified as “primarily waterfall.”

When do waterfall teams plan to adopt agile practices?



- 22% within the next year
- 28% within the next 2 to 4 years
- 50% do not plan to become agile.

Reasons given for this include satisfaction with current practice, a lack of knowledge about agile methods, and a lack of leadership to drive change.

How long ago did survey respondents who are now in agile or mixed environments report working in a primarily waterfall environment?



- 35% within the past two years
- 40% longer than two years ago
- 20% have only worked in Agile environments
- 5% Other response

2. Agile teams follow a wide variety of development practices.

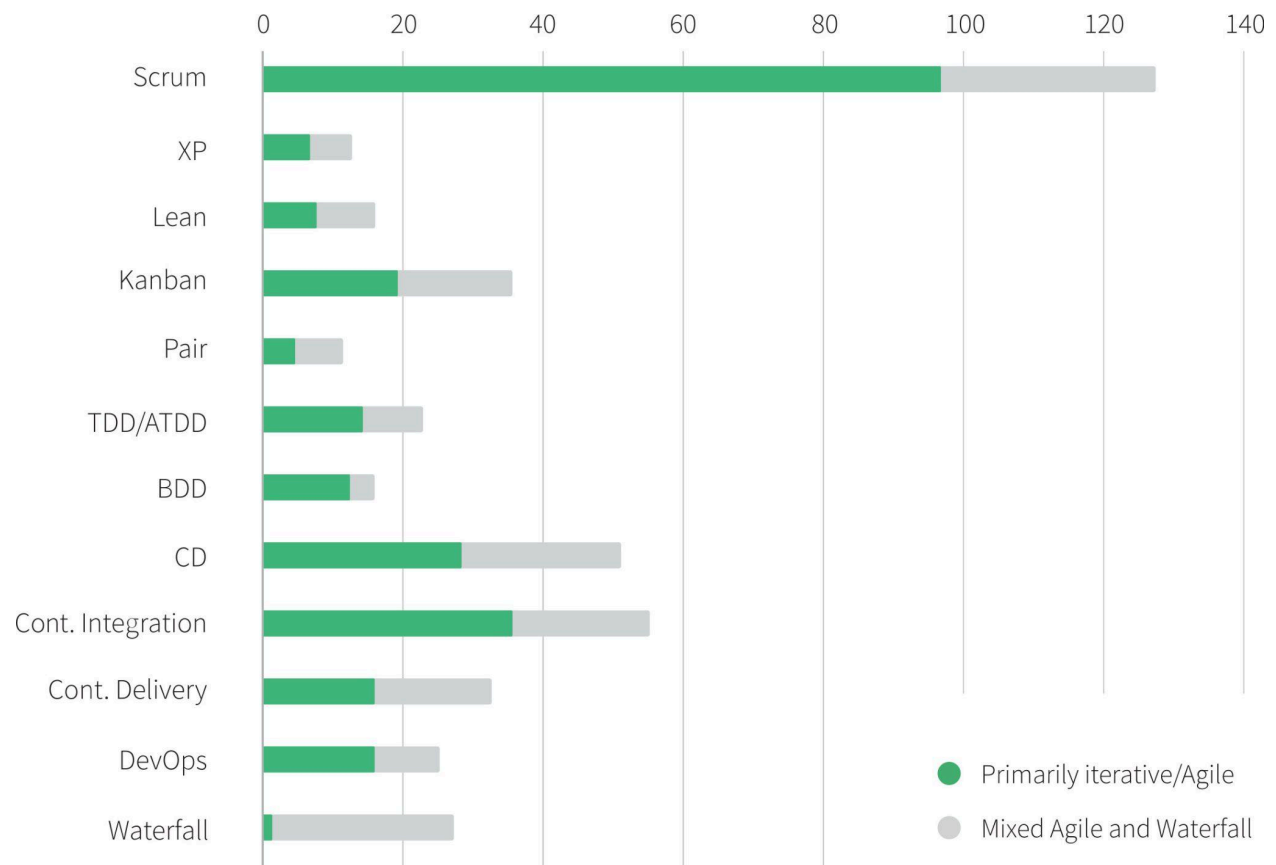
The goal of an agile team is to increase customer satisfaction through rapid delivery of a high-quality solution. Certain practices are common to all forms of agile:

- Close collaboration between all members of a project team, including developers, testers, business representatives, and customers

- Responsiveness to changing requirements
- Frequent releases of working software

Within these broad principles, there is wide variation in how teams practice agile development.

Ranorex Survey: Primary Development Practices



Scrum is the most popular development practice.

62% of respondents to the Ranorex survey chose Scrum as their primary practice, while 10% chose Scrum as their secondary practice. This is

consistent with the [2017 Stack Overflow Developer Survey](#), where 65% of developers reported using Scrum practices. The next-most popular primary practices are Continuous Integration (26%) and Kanban (17%).

Many teams use a combination of practices.

Respondents could choose any number of primary and secondary practices, or none at all. They reported employing an average of 1.8 primary practices and 2 secondary practices. Kanban is the most common secondary practice at 26%, followed closely by Test-Driven Development (TDD)/Acceptance Test-Driven Development (ATDD) at 25%.

Industry-wide, Kanban is used in combination with Scrum so often that the practice has its own name: [ScrumBan](#).

Almost half of teams follow continuous practices.

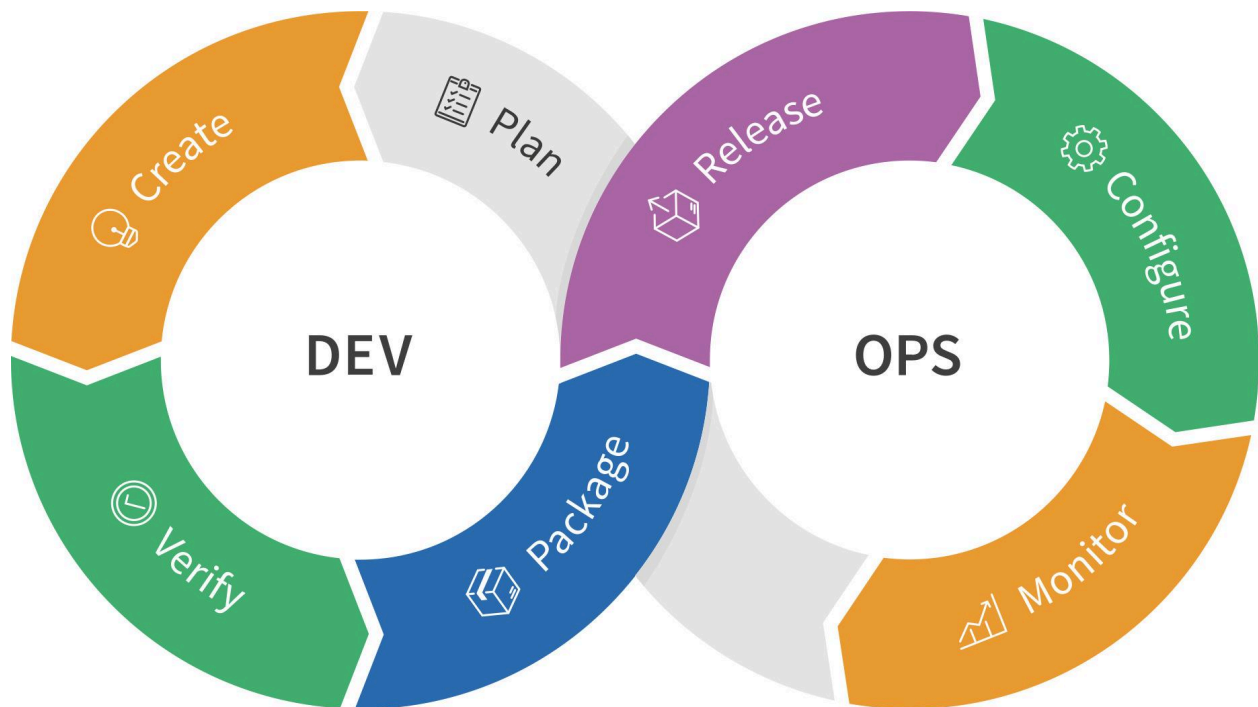
48% of respondents indicated that they use Continuous Integration (CI) practices either as a primary or secondary method. Developers on CI teams merge their code changes into a shared repository, and then a CI server such as Jenkins or Bamboo builds the application and runs automated integration tests. 31% of respondents use [Continuous Delivery](#) (CD), which extends CI by adding the automated execution of acceptance tests. CD ensures that there is a releasable version of the application available at all times. The application is not deployed to end users automatically, but is available as needed. Decisions to release software are driven by business factors rather than software development schedules.

“We...work in small focused teams following bits of Scrum, bits of Kanban, with a healthy dose of Extreme Programming.... Our cross functional teams have Product Management, all aspects of Product Development (including DevOps and Test), and UX all working together on a daily basis... The team works very closely, and like to pair program on pretty much everything.”

From a 2017 job posting for a software tester at a Fortune 500 company

3. Interest in DevOps is growing.

As adoption of CI/CD increases, system administrators and operators must respond to more frequent releases. To ensure that new software can be deployed successfully, organizations are adopting DevOps: an approach that brings together developers, testers, system administrators, and operators on the same team, ideally in a shared physical location. Responsibility for successful deployments is shared among all team members. The configuration and deployment processes are automated. DevOps can lead to improved communication and collaboration, faster release times and improved system stability.



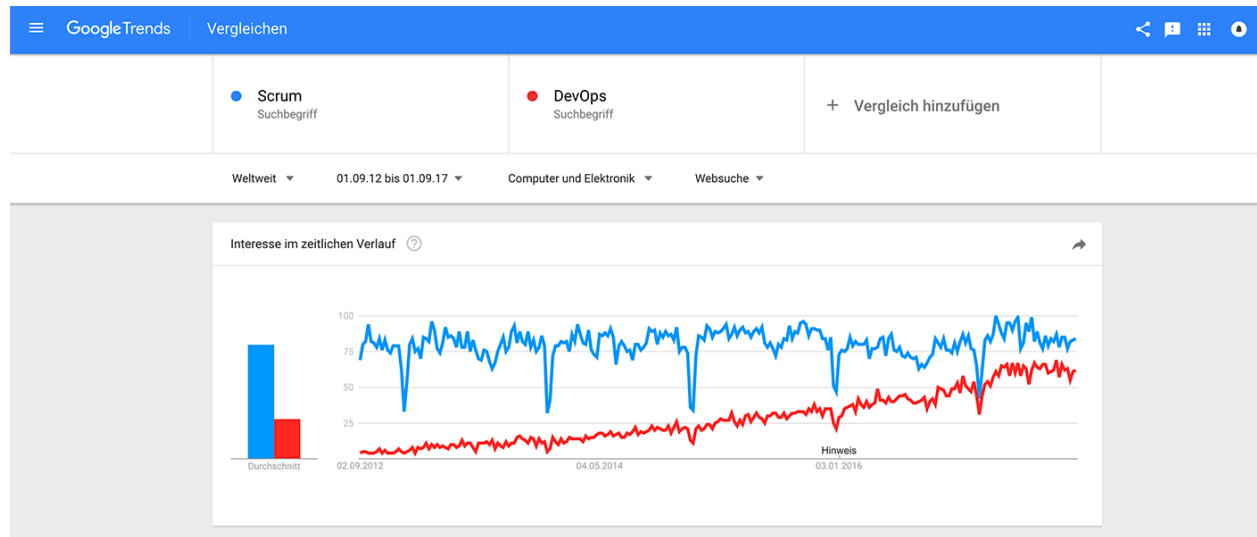
The DevOps Workflow

By Kharnagy – Own work, CC BY-SA 4.0,

<https://commons.wikimedia.org/w/index.php?curid=51215412>

In the Ranorex survey, 11% of respondents reported using DevOps as a primary development practice, and another 18% as a secondary practice. This is similar to the results in the [2017 Stack Overflow survey](#), where 11% of developers identified themselves as “DevOps specialists;” the same survey reported that DevOps specialists are the highest paid developers worldwide, well ahead of mobile and web developers.

A key indicator of interest in a subject is the number of related web searches. A Google Trends analysis shows that searches for the term “DevOps” have increased steadily since 2012, reaching their current peak in the summer of 2017. By comparison, interest in the term “Scrum” during the same period remained relatively unchanged.



Google Trends Comparison, DevOps vs Scrum

Interest in the terms “DevOps” (in red) vs “Scrum” (in blue) as measured by Google web searches in the “Computers & Electronics” category worldwide over the past 5 years. Scrum continues to lead but has remained relatively flat, while interest in DevOps has grown steadily. Trend results generated on 6 September 2017.

DevOps takes the principles of Agile and expands their scope, recognizing that ensuring high-quality development requires continual engagement and feedback from a variety of technical experts, including QA and operations specialists.

Aaron Cois

4. Testing is shifting left.

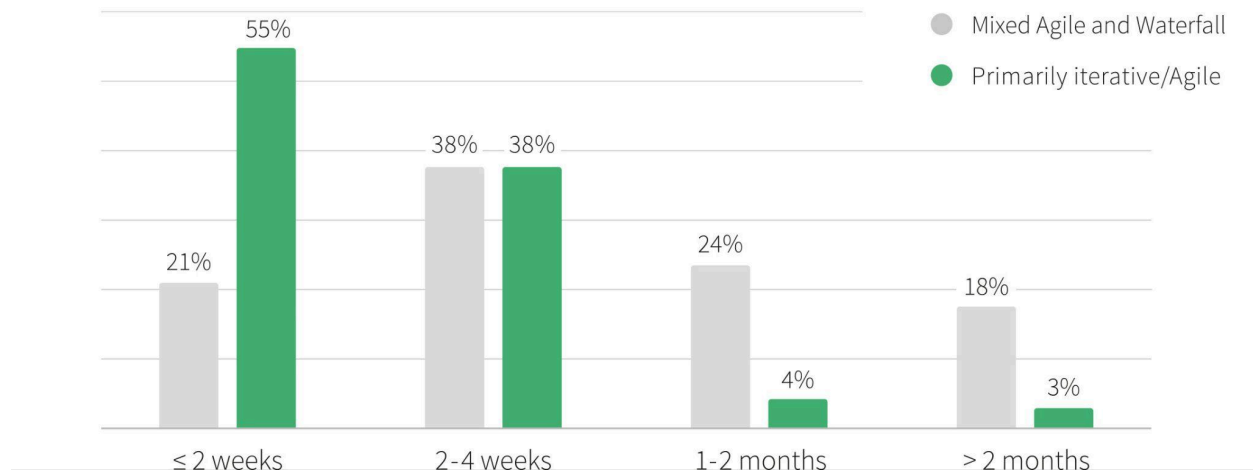
Test early, test often. It is well known that the cost of finding and resolving software defects rises as a software development project progresses. To reduce these costs, development teams are testing as early as possible in the development process – shifting the overall testing effort from the right side of the project timeline to the left.



35% of respondents to the Ranorex survey shift left by creating tests before coding begins, applying the practices of Test-Driven Development (TDD) or Acceptance Test-Driven Development (ATDD). TDD and ATDD are [strategies](#) for designing and building quality into an application, so that teams can spend less time resolving defects — time that is critical in short development iterations.

Iteration Length

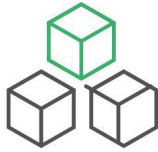
Primarily agile projects vs. mixed agile/waterfall



As might be expected, respondents in primarily agile environments do have shorter development iterations. In the Ranorex survey, 55% of respondents in primarily agile environments reported iterations of two weeks or less as compared to 21% in mixed agile/waterfall environments. These shorter windows require fast feedback from testing.

The shortest iterations are found in CI/CD and DevOps environments, where new code may become available for testing every day – or even several times per day. Testing in continuous environments presents unique challenges. First, tests must be ready to run as soon development completes, which means that testers might begin writing their tests while developers are still working on the code. Tests may have to be re-written quickly in response to changing requirements. In addition, automated tests must complete in a very short time frame, so it may be necessary to execute them on multiple remote systems, in parallel.

Given these challenges, [continuous testing environments](#) benefit from a combination of automated and manual tests, as shown below.



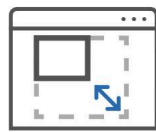
Automated unit and interface/API tests



Manual usability and exploratory testing by QA professionals⁷



Automated acceptance tests



Manual and/or automated non-functional tests, such as performance and security tests



Automated integration, regression and system tests performed during system build

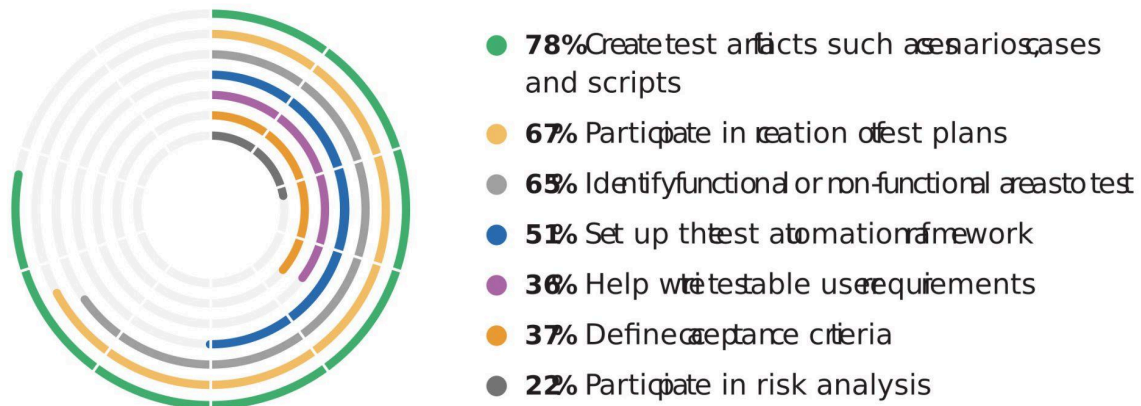
The practice of testing during development means that the separation of roles between testers and developers is fading. As a result, many employers prefer testers who have programming or test automation experience. For more information on the effect of agile development methodologies on job requirements for testers, refer to sections 7, 8 and 9, below.

5. Teams benefit when testers collaborate with developers.

In agile practice, a cross-functional team collaborates from the start of the software development lifecycle to build quality into the product and prevent defects before they occur. Ideally, there should be testable acceptance criteria for each user story before development begins; and an agreement that a story does not meet the “definition of done” until it has passed the acceptance criteria .

Ranorex Survey: QA/Tester Planning Responsibilities

What planning tasks do testers on agile teams do regularly?



Tester collaboration during planning

In the Ranorex survey, respondents on agile teams reported that they commonly create test artifacts such as test cases, participate in the creation of test plans, and help identify areas to test. However, fewer respondents said that they were able to help write testable user requirements or define acceptance criteria. On fully agile teams, these numbers should be closer to 100%.

Ranorex Survey: QA/Tester Project Responsibilities

What project tasks do testers on agile teams do regularly?



- 54% Create automated GUI test cases
- 53% Analyze completed tests
- 52% Conduct exploratory GUI testing
- 50% Document/archive tests for regression testing
- 46% Conduct scripted GUI testing
- 34% Create post-testing artifacts such as coverage
- 22% Work with developers to conduct unit interface/API testing
- 11% Coach developers in testing techniques

Tester collaboration during development

During development, testers and developers on agile teams should work in pairs to ensure that effective unit testing is occurring; and that testers understand the product and its requirements to support exploratory testing. However, only 22% of respondents to the Ranorex survey reported that they assist developers in this way. Just 11% of respondents regularly coach developers in testing techniques. More commonly, testers create automated test cases (54%), conduct exploratory testing (51%) or scripted testing (46%), and analyze completed tests (53%).

Our experience with agile teams has shown that testing skills and experience are vital to project success, and that testers do add value to agile teams.

Lisa Crispin & Janet Gregory

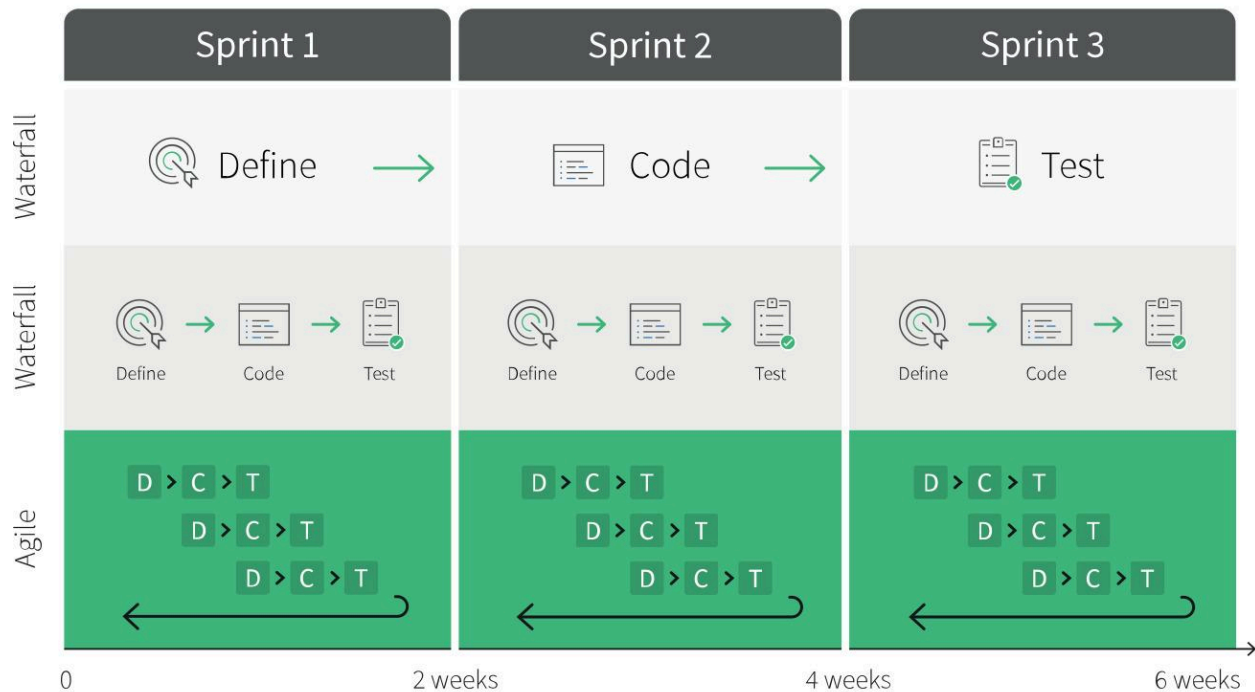
6. Team organization is important for success.

Respondents to the Ranorex survey reported several benefits of testing in agile environments:

- “Shorter cycles means smaller code, which means the potential of finding and correcting issues faster.”
- “[Allows] more frequent testing – including updating of tests.”
- “Constant/continuous feedback...helps to keep developers on track.”
- “Testing can be done in a timely manner during the sprint; changes can be integrated quickly.”

However, some teams are dissatisfied with their current practices.

35% of respondents to the Ranorex survey are in “mixed agile and waterfall environments.” Testing can be negatively affected when a sprint becomes a mini-waterfall, with activities occurring in shortened but still structured phases. Testers may have nothing to do at the start of the sprint, and then be rushed at the end of the sprint. Or, testing may be delayed until the start of the next sprint. To avoid this issue, system and acceptance testing should occur as each story is completed, rather than for all stories at the end of a sprint – or worse, in a special “testing” sprint, as shown below:



Concerns expressed in the Ranorex survey included:

- Resistance to change when large parts of the organization use waterfall methods
- Testing pushed to the end of a sprint, so there is insufficient time for testing
- A lack of management understanding regarding agile practices
- Waterfall-style separation of roles into 'QA' and 'Dev', rather than a whole-team approach.

Best practice calls for software testers to be full members of the team.

Many waterfall practices don't work well in combination with an agile environment. Foremost among these is keeping QA and development as separate groups. For an agile environment to be successful, testers should function as full members of the development team. QA should contribute to sprint planning so that there are adequate resources for testing, participate

in customer/business meetings, help develop testable story requirements, and partner closely with developers throughout the iteration window .



In a “three amigos” meeting, a product owner, developer, and tester work together to create testable user stories and acceptance criteria.

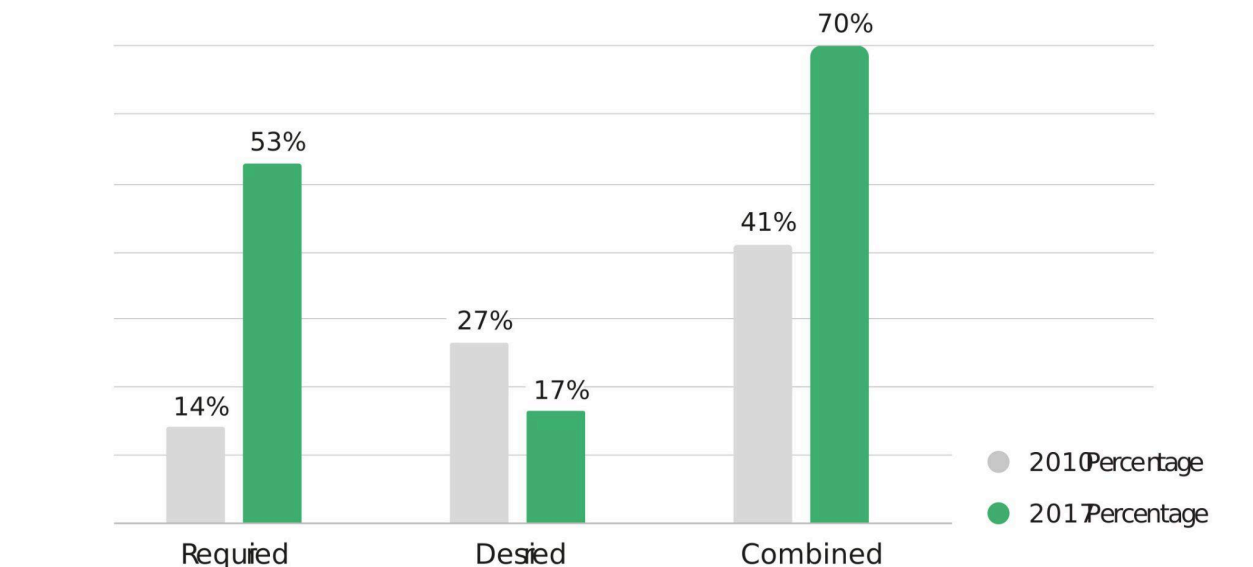
Ryan Thomas Hewitt

7. More employers want test automation experience.

In 2010, agile testing expert Elisabeth Hendrickson published the [results of her research](#) into job requirements for software testers or QA staff in the USA . To determine how these requirements may have changed in the past

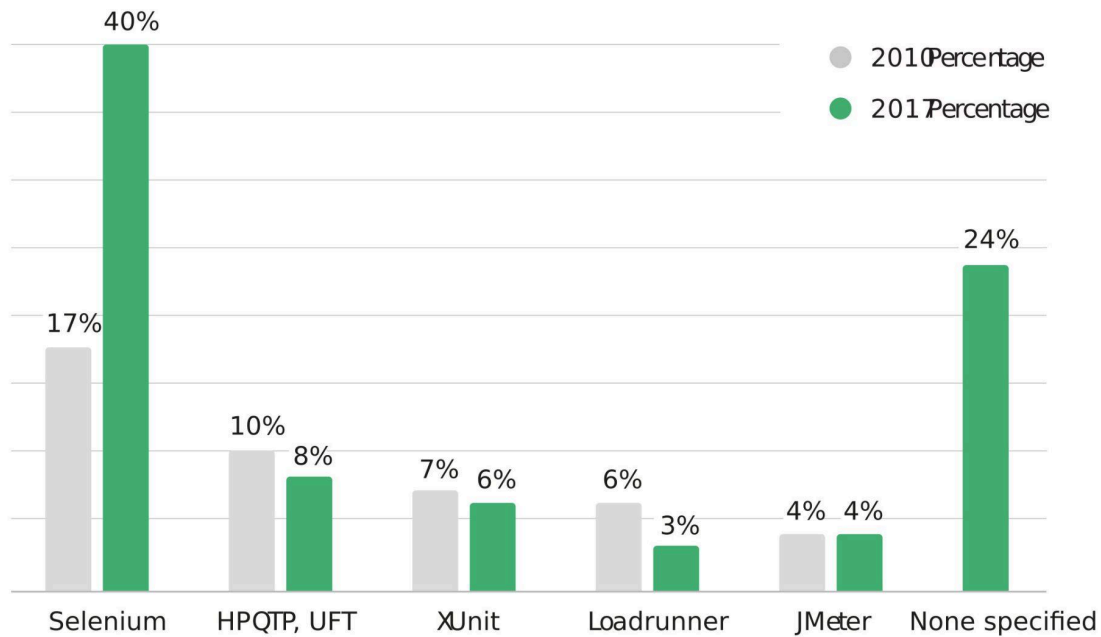
seven years, Ranorex reviewed 155 online job postings using the search terms “Software Tester” and “QA Tester,” following Hendrickson’s original methodology as much as possible. The results show a dramatic increase in demand for test automation skills.

Employment Research: Test Automation Experience



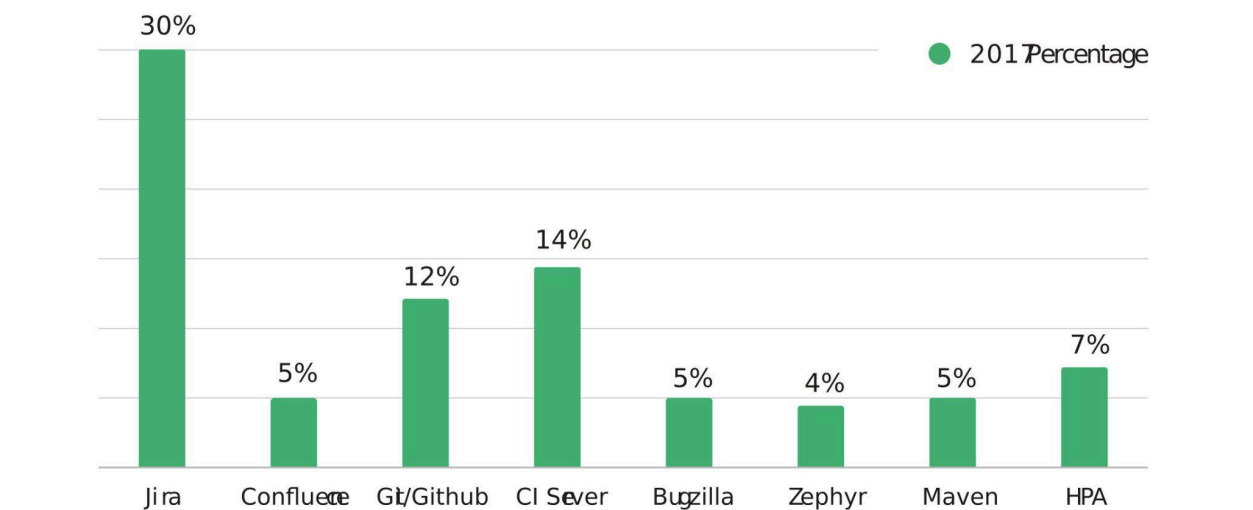
In 2010, only 14% of job postings required test automation experience. By August 2017, that number had risen to 53%, with another 17% of potential employers expressing a preference for automation experience.

Employment Research: Top Requested Automation Technologies



The top test automation experience requested in both 2010 and 2017 is Selenium. Interestingly, in the 2017 review of job postings, 24% of employers expressed a preference for experience with any test automation tool.

Employment Research: Top Requested Automated Test Management Tools



In 2017, the top requested automated test management tool is Atlassian Jira at 30%, followed by CI servers such as Jenkins, Hudson, and Bamboo at 14%. The 2010 survey did not include results for automated test management tools.

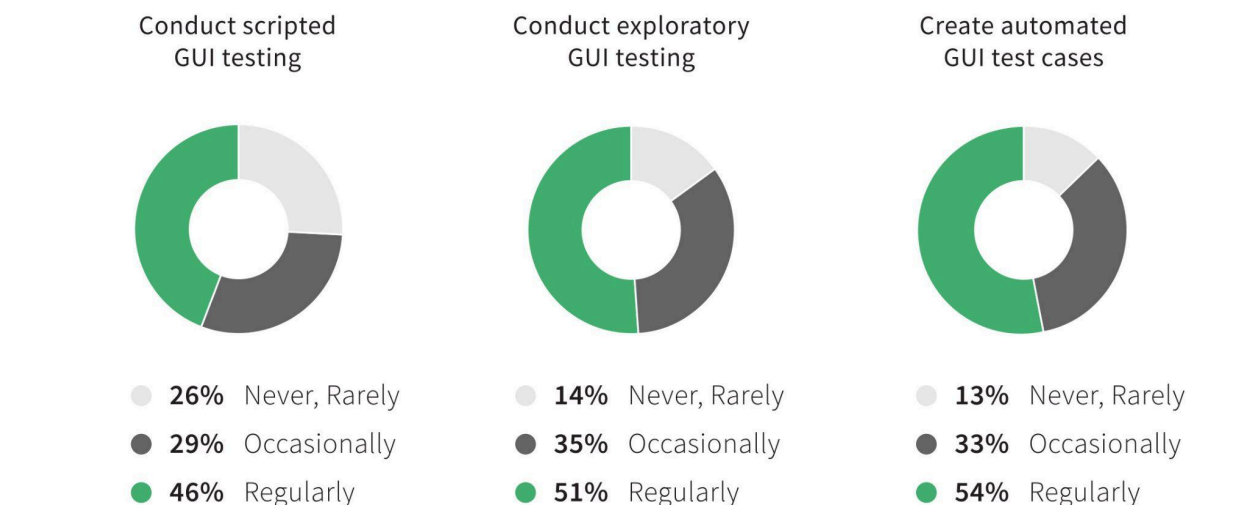
8. Traditional approaches to testing are still valued.

Most teams see a need for GUI testing.

The increasing emphasis on unit and integration testing sometimes [raises questions](#) about the [relative importance](#) of traditional GUI testing. However, the ideal mix of testing for a given project depends on the nature of the application and the level of risk. A mobile app may need a higher volume of automated GUI tests relative to unit and API tests since much of its functionality is in the GUI. In addition, unit and integration testing don't address important aspects of user experience (UX) and usability testing.

Ranorex Survey: GUI Testing

Software testers on teams that are primarily agile:



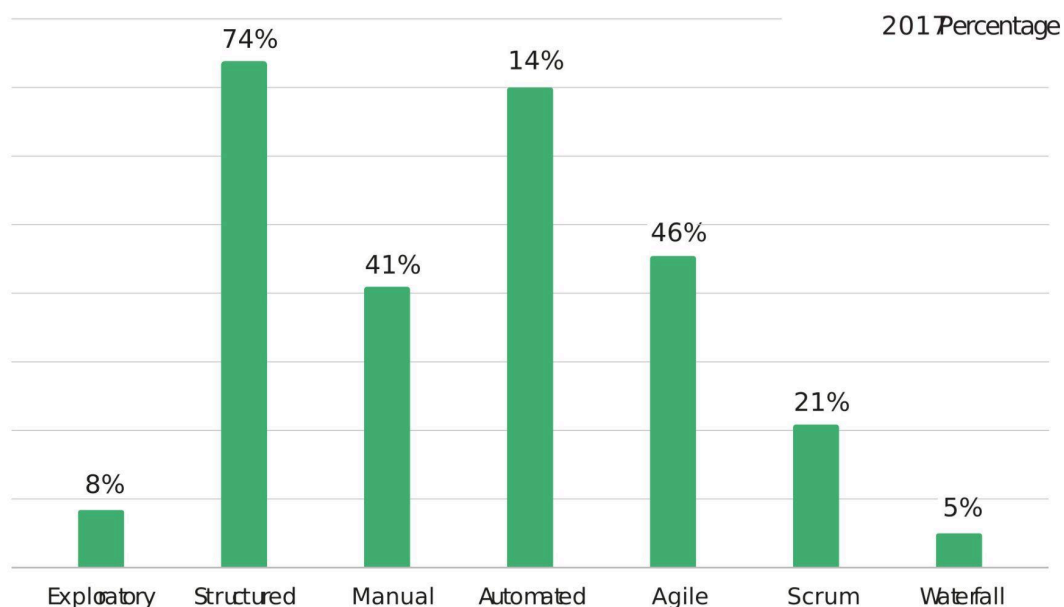
On teams that are primarily agile, 75% of software testers in the Ranorex survey occasionally or regularly conduct scripted testing of the user

interface, while 85% occasionally or regularly do exploratory testing of the user interface. Both types of testing may be completely manual or may be assisted by test automation. 86% of respondents to the Ranorex survey indicate that the testers on their teams occasionally or regularly create automated GUI test cases.

Employers are looking for traditional skills.

While 70% of positions asked for automated testing experience, 41% specifically mentioned “manual testing” as one of the testing skills either required or preferred. 74% of positions listed structured testing experience, such as experience in writing and executing test cases or test scripts. These results demonstrate that even as the separation of roles between developer and tester is decreasing, there is still strong demand for traditional testing skills.

Employment Research: Types of Testing Experience Desired or Required



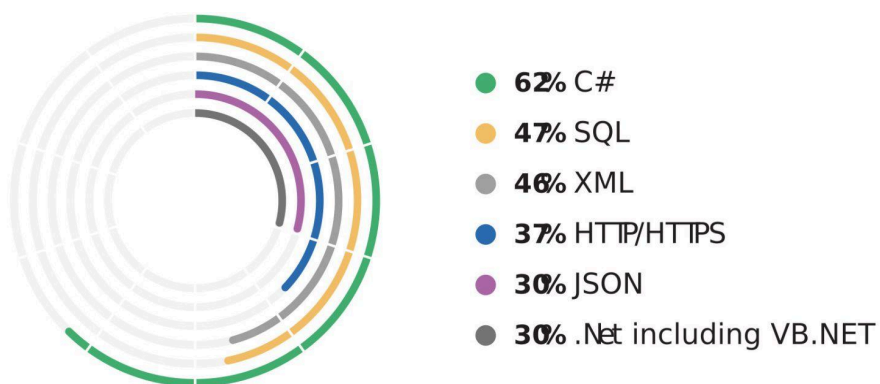
To be counted as “manual” or “automated”, an advertisement had to specifically include one of these terms. For “structured,” the advertisement had to include one of the following terms: “test case,” “test script,” or “scripted”. For “exploratory,” the term “ad hoc” also counted.

9. Fewer job advertisements are requiring programming skills for testers.

Ranorex survey respondents were asked, “which languages do the software testers on your team need to know, if any?” The top responses were C# at 62%, followed by SQL at 47% and XML at 46%.

Ranorex Survey: Top Languages

Question: which languages do the software testers on your team need to know, if any?

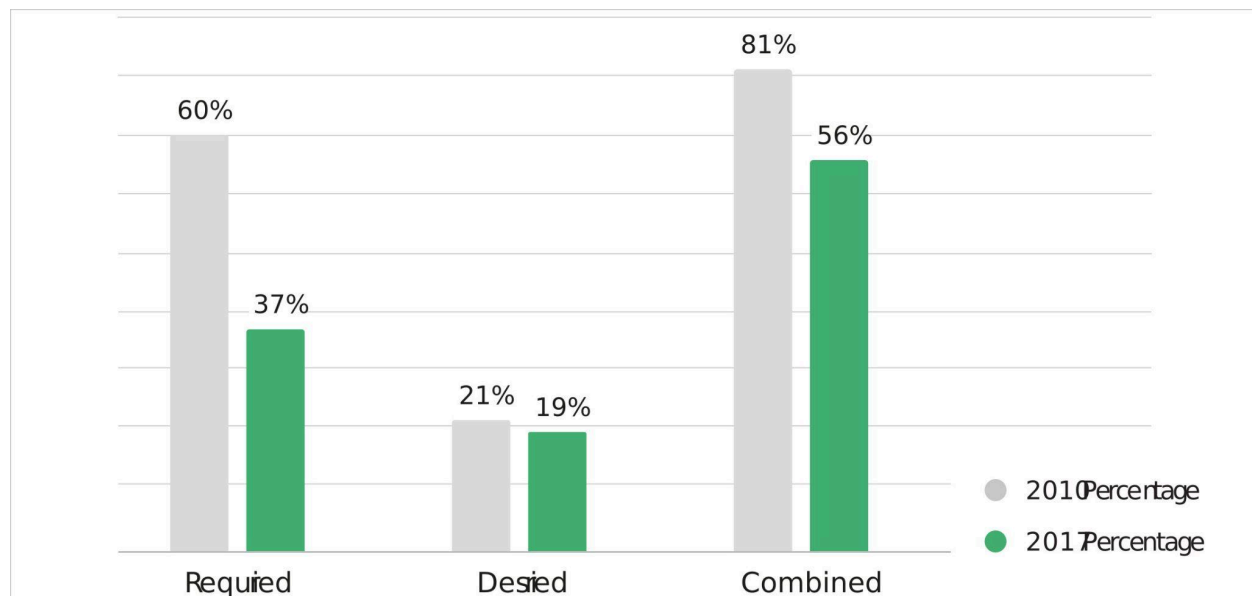


While Ranorex Studio supports all testing applications written for all major platforms, Ranorex code modules are written in either C# or VB.NET, which is reflected in the inclusion of these languages in the survey results. Of course, Ranorex Studio also offers fully codeless creation of tests, so testers without programming skills can build automated tests.

As software testers work more closely with developers, having programming skills is an asset. Therefore, it seems surprising that a review of job advertisements in 2017 showed a decrease in the number of

employers requiring programming skills for testers, from 60% in 2010 to 37% in 2017. However, this decrease in requirements for programming experience occurred at the same time as an even greater increase in requirements for test automation experience. It is likely that these two changes are related.

Employment Research: Programming Experience for Testers



The language most requested by potential employers is SQL. In 2010, 45% of potential employers requested experience with SQL. Requests for SQL dropped slightly in 2017, to 41% of potential employers. Java was the next-most requested language in both 2010 and 2017.

Key Takeaways

In an industry that is rapidly moving toward continuous testing:

- Turnaround times are decreasing.
- Fast feedback is essential.
- While the line between the role of developer and tester is blurring, traditional testing skills are still in demand.

- Requirements for test automation skills are increasing.

Wondering how to get started with test automation? Ranorex can help.

With Ranorex, test automation is available to coders and non-coders alike. If you are already using Ranorex Studio, you know that our matchless object recognition features and codeless scripting make test automation accessible to non-coders. With only some C# or VB.NET coding skills, testers can customize automation scripts in Ranorex, while those with full C# or VB.NET coding skills can create fully-customized automation scripts. Ranorex also integrates with essential web technologies including Selenium WebDriver and Selenium Grid, HTML5, Adobe Flash and Flex, and supports testing of web applications written in languages such as Java and Javascript. In addition, Ranorex offers a SQL data connector that allows you to build SQL queries and preview their results.

Download a free trial version of Ranorex today to experience the power of test automation. Or, register for one of our free, informative webinars to learn more.

[**Download a free trial of Ranorex Studio**](#)

[**Register for a free, live webinar**](#)

Employment Research Methodology

As in the original 2010 research by Hendrickson, Ranorex used only postings from direct hire jobs, screening out jobs from placement firms and those without an identifiable employer. All postings were accepting applications online between 1 August 2017 and 31 August 2017, and were retrieved from a variety of websites including LinkedIn, Indeed.com, Monster.com and Dice.com. 69% of positions were for placement in the USA, 32% in Europe and 8% in other locations worldwide. Care was taken to eliminate duplicate postings that appeared on multiple sites. Jobs that included the word “test” but primarily seemed to be programming were also excluded. No attempt was made to search for or exclude “Test Automation”

positions. However, these types of positions occasionally appeared in searches using keywords such as “Software Tester” and “Software QA.” Of the total number of jobs analyzed for these keywords, 6% were primarily test automation. Jobs were treated as requiring programming experience if they stated that a specific language was required, or if they stated that programming was required without identifying a specific language. However, a request for experience in testing systems written in a specific language was not treated as a requirement for the ability to program in that language.